```
[15] from google.colab import drive
     drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

Import Essential Tools

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import re
import string

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
import nltk

nltk.download('stopwords')
nltk.download('punkt_tab')
nltk.download('wordnet')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
True
```

Text Classification Exercise

Load Dataset

```
[17] df = pd.read_csv("/content/drive/MyDrive/Artificial Intelligence and Machine Learning/Week8Workshop8/trum_tweet_sentiment_analysis.csv")
```

```
[18] df.columns
```

```
Index(['text', 'Sentiment'], dtype='object')
```

```
[19] assert 'text' in df.columns and 'Sentiment' in df.columns, "Dataset must contain 'text' and 'sentiment' columns."
```

Cleaning and Tokenization

Helper Functions

```
[20] def lower_case(text):
       return text.lower()
```

```
[21] def remove_url(text):
       return re.sub(r"http\S+|www\S+|https\S+", '', text, flags=re.MULTILINE)
```

```
[22] def remove_mentions(text):
       return re.sub(r'@\w+', '', text)
```

```
[23] def remove_punctuations(text):
       return text.translate(str.maketrans('', '', string.punctuation))
```

```
[24] def remove_stopwords(tokens):
       stop_words = set(stopwords.words('english'))
       tokens = [word for word in tokens if word not in stop_words and word.isalpha()]
       return tokens
```

```
[25] def lemmatize_words(tokens):
       lemmatizer = WordNetLemmatizer()
       tokens = [lemmatizer.lemmatize(token) for token in tokens]
       return tokens
```

```
[26] def stemm_words(text):
       porter = PorterStemmer()
       stemm_tokens = []
       for word in text:
         stemm_tokens.append(porter.stem(word))
       return stemm_tokens
```

Build a Text Cleaning Pipeline

```
[27] def text_cleaning_pipeline(text, rule = "lemmatize"):
       text = lower_case(text)

       text = remove_url(text)

       text = remove_mentions(text)

       text = remove_punctuations(text)

       tokens = word_tokenize(text)

       tokens = remove_stopwords(tokens)

       tokens = lemmatize_words(tokens)

       return " ".join(tokens)
```

```
[24] def remove_stopwords(tokens):
       stop_words = set(stopwords.words('english'))
       tokens = [word for word in tokens if word not in stop_words and word.isalpha()]
       return tokens
```

```
[25] def lemmatize_words(tokens):
       lemmatizer = WordNetLemmatizer()
       tokens = [lemmatizer.lemmatize(token) for token in tokens]
       return tokens
```

```
[26] def stemm_words(text):
       porter = PorterStemmer()
       stemm_tokens = []
       for word in text:
         stemm_tokens.append(porter.stem(word))
       return stemm_tokens
```

Build a Text Cleaning Pipeline

```
[27] def text_cleaning_pipeline(text, rule = "lemmatize"):
       text = lower_case(text)

       text = remove_url(text)

       text = remove_mentions(text)

       text = remove_punctuations(text)

       tokens = word_tokenize(text)

       tokens = remove_stopwords(tokens)

       tokens = lemmatize_words(tokens)

       return " ".join(tokens)
```

Evaluation

```
[32] y_pred = model.predict(X_test_tfidf)

     print("Classification Report:\n")
     print(classification_report(y_test, y_pred))
```

```
Classification Report:

              precision    recall  f1-score   support

           0       0.93      0.95      0.94    248842
           1       0.90      0.86      0.88    121183

    accuracy                           0.92    370025
   macro avg       0.92      0.91      0.91    370025
weighted avg       0.92      0.92      0.92    370025
```

```
[33] cm = confusion_matrix(y_test, y_pred, labels=model.classes_)
     plt.figure(figsize=(6, 4))
     sns.heatmap(cm, annot=True, fmt='d', cmap="Blues", xticklabels=model.classes_, yticklabels=model.classes_)
     plt.title("Confusion Matrix")
     plt.xlabel("Predicted")
     plt.ylabel("Actual")
     plt.tight_layout()
     plt.show()
```